자율주행미들웨어응용

Week 0 — ROS Fundamentals

Jaerock Kwon, PhD
Associate Professor
Electrical and Computer Engineering
University of Michigan-Dearborn

Today

No lab. Nothing due. Nothing graded.

The point of today is to tell you what the next fifteen weeks ask of you,
and to give you the vocabulary the rest of the course assumes you have.

If you registered late — **this page is your starting point.**
It is written to stand alone.

1. What this course is

You learn ROS 2 by finishing a real vehicle

MRider: a 12 V ride-on car becoming a drive-by-wire autonomous platform.

- The simulator already drives, maps, and navigates
- The **vehicle does not exist yet** — parts are being ordered
- Every deliverable closes a named gap in a real project

Not exercises.

What does not exist today

Missing	Status in the repository
`firmware/`	No firmware at all
`mitt_hardware`	The plugin that makes sim and real the same stack
Measured geometry	**"NOTHING IN THIS FILE HAS BEEN MEASURED"**
Real tests	29 tests exist. All 29 are linters

By December these exist because **you** built them.

The semester

Weeks		
0	Today	Orientation — no lab
1–3	Foundations	Individual labs, everyone the same
4	**No class** — 개천절	Self-study + Lab 4
5	Description to motion	Lab 5
6–10	Four tracks	Electrical · Chassis · Simulation · Software
11–14	Merge ladder	Subsystems combine, pairwise
15	**Final demo**	12/21

Honest failure outscores a lucky success.

A subsystem that does not work, with an evidenced diagnosis of **why**, scores above one that works for reasons you cannot explain.

2. What a robot is

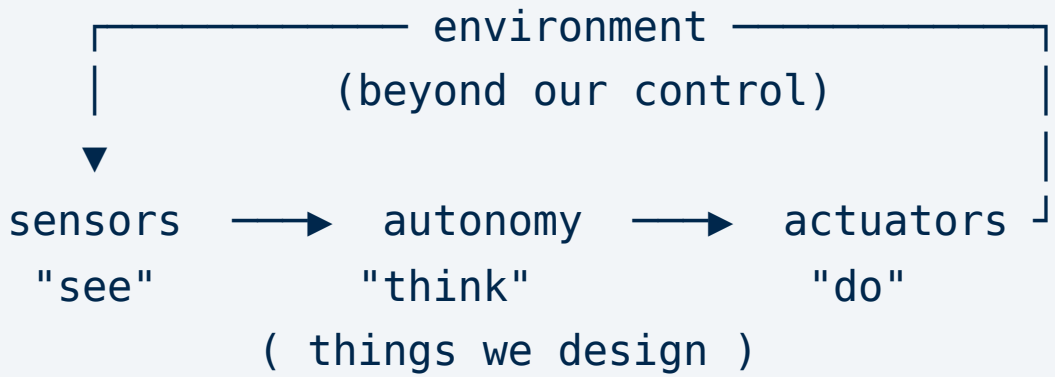
Three definitions that disagree

Oxford: a machine — especially one programmable by a computer — capable of carrying out a complex series of actions automatically.

IEEE: an autonomous machine capable of **sensing** its environment, carrying out **computations** to make decisions, and performing **actions** in the real world.

Engelberger: "I don't know how to define a robot, but I know one when I see one."

Sense → think → act



The split matters more than the diagram.

Everything inside the robot is yours to design and be responsible for.

The environment is not.

MRider in that loop

See	2D LiDAR · camera · IMU · wheel encoder · steering angle sensor
Think	SLAM · EKF · Nav2 — mapping, state estimation, planning, control
Do	Steering gearmotor · two drive motors through a motor driver

Most of the hard problems in this course live at the boundary —
where a designed thing meets a world that does not cooperate.

3. The tools

You cannot build complex systems without them

Linux — open source, mature, what robots actually run.

We use **Ubuntu 22.04 LTS, installed natively.**

Not a VM. Not WSL.

Gazebo needs a real GPU, and that is the whole reason.

New to Linux? `ryanstutorials.net/linuxtutorial` — work through it this week.

The notation every lab uses

```
$ <command> [options]
```

`$`	The prompt. Indicates a terminal command — you do not type it
`<...>`	**Required.** Replace with your own value
`[...]`	**Optional**

So `$ ros2 topic echo <topic_name>` means: type `ros2 topic echo` followed by an actual topic name.

You will use **bash**.

git

Tracks versions of code, and shares it.

You will submit work through it — and your **commit history is part of your participation grade.**

Not as surveillance. Because **what you did and when** is genuinely the best available record of engineering work.

Learn these six before Week 2:

`clone` · `status` · `add` · `commit` · `push` · `pull`

Why not Docker

Containers package an entire configuration so it runs identically everywhere. Reasonable in general — and the reference course offers it.

This course does not use it.

You will run Gazebo for most of the semester. It needs **GPU-accelerated OpenGL** — which works acceptably in Docker on a Linux host and badly everywhere else.

Debugging GPU passthrough costs more time than a native install ever will.

4. What middleware is for

A robot is not one program

Camera driver · LiDAR driver · localiser · planner · controller · safety monitor

Different authors. Different languages. Different rates.
Sometimes different machines.

They have to talk.

Old days: everyone wrote their own code… everyone sorted out multi-process communication on their own (poorly).

ROS2 Deep Dive, EECE 5560

What ROS 2 gives you

- **Named, typed channels** instead of ad-hoc sockets
- **Discovery** — processes find each other unaided
- **Introspection** — list, inspect, record, replay at runtime
- **Language independence** — Python and C++ in one system

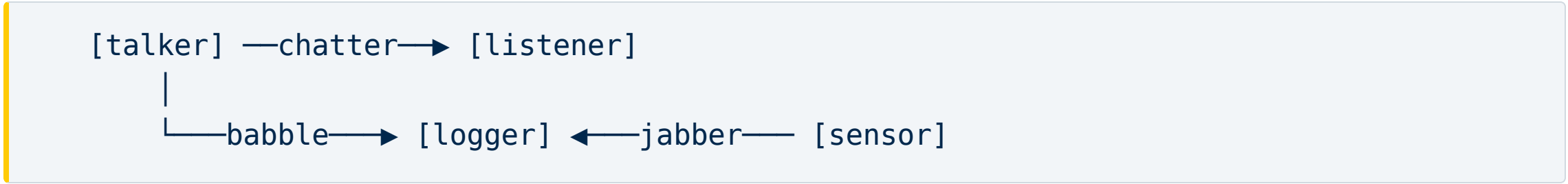
What it costs you

Coupling that used to be a **compile error**
is now a **runtime condition.**

- Topic names must agree
- Message types must agree
- QoS settings must agree
- Domain IDs must agree

When they disagree, **mostly nothing happens, and nothing complains.**

The graph



No node knows where the others are, or whether they exist.
It publishes to a **name**, and anyone listening receives it.

That is what makes the parts independently replaceable —
and why a typo in a topic name produces **silence, not an error.**

Underneath: DDS

ROS 2 does not implement its own networking.

It sits on **DDS** — Data Distribution Service, an industry standard for real-time message passing, with several competing implementations.

You mostly do not need to care.

Except that MRider pins one specific implementation,
and the reason is a real bug. **That is a Week 1 story.**

5. The vocabulary

Code

Term	What it is
Node	A single program in the graph. One job each, by convention
Package	A unit of distribution: nodes, configs, launch files, messages
Workspace	A directory of packages you build together with `colcon`

Communication

Term	What it is	Use when
Topic	Named, one-way, many-to-many stream	Continuous data
Message type	The schema on a topic	Both ends must agree, exactly
Service	Request → response, blocking	A quick, discrete question
Action	Request → feedback → result	Slow, and cancellable

State and geometry

Parameter — named, typed, per-node configuration.
Readable **and settable at runtime**.

Transform (TF) — the relationship between coordinate frames, over time.

Every sensor reading arrives in **some** frame. The LiDAR reports distances from the LiDAR. The camera sees from the camera.

Turning that into **"where is the obstacle relative to the car"** is what TF is for — and getting it slightly wrong gives you a system where every component is healthy and **the answer is quietly wrong.**

6. Humble, not Jazzy

Why the older release

Distribution	Ubuntu	
Humble Hawksbill	22.04	**This course**
Jazzy Jalisco	24.04	What most tutorials assume

Because that is what the **working** MRider stack runs on. The simulator, controllers, SLAM and Nav2 are all verified on Humble.

Choosing the newer release would mean porting and re-verifying a working system before the course could even start.

So most tutorials will disagree with us

Including the recommended textbook.

Most content transfers directly. Three areas differ enough to waste your afternoon:

- `ros_gz` package naming
- the `gz_ros2_control` build
- several Nav2 parameter names

Where a weekly note and an external tutorial disagree, follow the weekly note.

7. Before Week 1

Week 1 opens with a lab

It assumes a working machine. Getting there takes **2–4 hours**, most of it unattended downloading.

- [ ] Read **Environment Setup** end to end **before** starting
- [ ] Install Ubuntu 22.04 LTS **natively**
- [ ] ROS 2 Humble + Gazebo Harmonic
- [ ] Clone the repo, build the workspace
- [ ] `bash scripts/check_env.sh` until it reports `FAIL: 0`
- [ ] Collect your `ROS_DOMAIN_ID`, put it in `~/.bashrc`

Start the graphics step early

Everything else on that list is downloading and waiting.

The one step that genuinely **fails** is GPU drivers.

If `glxinfo -B` reports `llvmpipe`, Gazebo runs at about one frame per second and nothing in this course works.

That is not fixable on the morning of class.

And if you get stuck

Stuck for more than **30 minutes**? Stop. Save the terminal output.
Bring it, or post it.

Debugging an install alone at 2 a.m. is the least productive form of learning available to you — and there is a great deal of real work waiting.

Next week: middleware, drive-by-wire, and Lab 1.